

CO4

ASSESSMENT TOOL-1

NAME : P. Ranjithkumar

DATE:14.08.2026

REG.NO : 192411119

Friday

COURSE CODE : CSA1103

COURSE NAME : OOAD

Q1. Architectural Patterns in Object-Oriented Systems

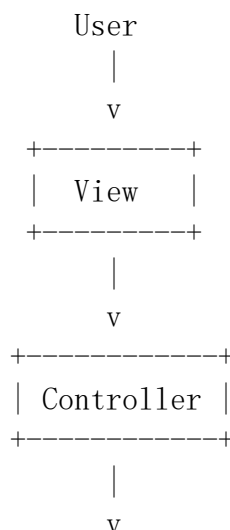
Definition

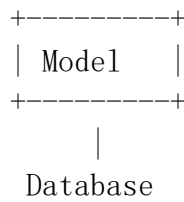
Architectural patterns are high-level design solutions that define the overall structure of a software system. They organize components and describe how they interact.

Common Architectural Patterns

1. Model-View-Controller (MVC)
2. Layered Architecture (Three-Tier)

MVC Architecture





Components

Model

- Stores application data.
- Contains business logic.

View

- Displays data to users.
- Provides graphical interface.

Controller

- Receives user requests.
- Updates Model and View.

Advantages

- High modularity
- Easy maintenance
- Reusable components
- Supports parallel development
- Easy testing

Conclusion

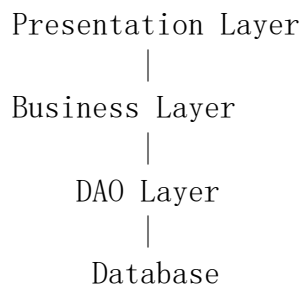
MVC separates data, interface, and control logic, making applications flexible and maintainable.

Q2. Data Access Object (DAO) Pattern

Definition

DAO is a design pattern that separates database access code from business logic.

Diagram



Working

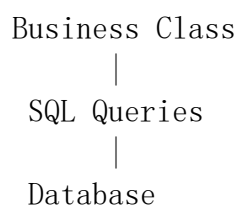
1. User sends request.
2. Business layer calls DAO.
3. DAO executes SQL queries.
4. Database returns results.
5. DAO sends data back.

Advantages

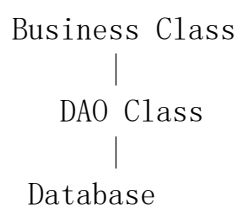
- Database independence
- Easy maintenance
- Code reusability
- Easier testing
- Centralized database operations

Example

Without DAO



With DAO



Conclusion

DAO keeps database code separate from application logic, improving maintainability.

Q3. Object-Oriented Design Guidelines

Definition

Design guidelines are principles used to develop reusable, maintainable, and scalable software.

Categories

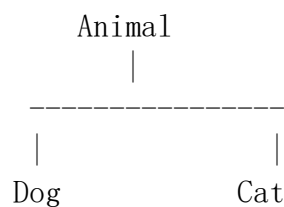
1. SOLID Principles
2. GRASP Principles

Liskov Substitution Principle (LSP)

Definition

A subclass should be able to replace its parent class without affecting program correctness.

Diagram



Both Dog and Cat should work wherever Animal is used.

Benefits

- Better inheritance
- Code reuse
- Easy extension
- Less maintenance
- Reliable software

Conclusion

LSP ensures subclasses behave like their parent class, improving software quality.

Q4. Database Persistence

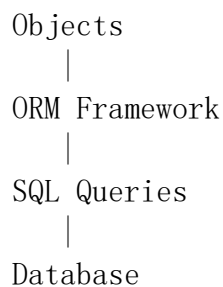
Definition

Persistence means storing object data permanently in databases.

Persistence Mechanisms

1. JDBC
2. ORM (Hibernate, JPA)

ORM Diagram



Working

- Object created
- ORM converts object to table row
- Data stored automatically
- Retrieves objects from database

Advantages

- Less SQL coding
- Automatic mapping
- Easy maintenance
- Better productivity

Conclusion

ORM simplifies persistent object management by mapping objects to database tables.

Q5. Application Layer

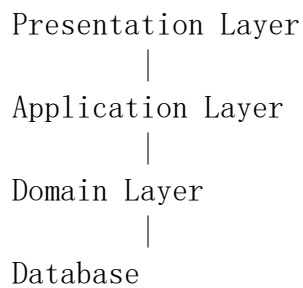
Definition

Application Layer coordinates application activities and executes use cases.

Responsibilities

- Controls workflow
- Coordinates business operations

Diagram



Working

- Receives user request
- Calls domain services
- Returns response

Advantages

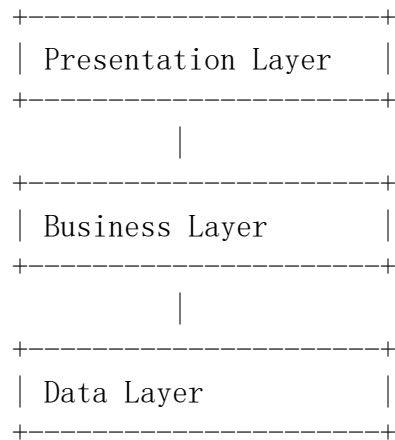
- Business rules remain in Domain Layer
- Better organization
- Loose coupling
- Easy maintenance

Conclusion

Application Layer manages use cases without containing business logic.

Q6. Three-Tier Architecture

Structure



Customer Transaction Flow

Customer



Presentation Layer
(Login/Product Selection)



Business Layer
(Validation, Payment, Order Processing)



Data Layer
(Database Update)



Business Layer



Presentation Layer



Customer receives confirmation

Advantages

- Better security
- Easy maintenance
- Scalability
- Reusability

Conclusion

Three-tier architecture separates responsibilities, making e-commerce systems efficient.

Q7. Business Logic Layer

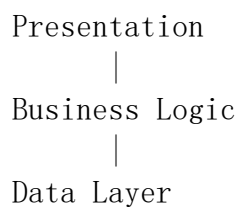
Definition

Business Logic Layer contains application rules and decision-making processes.

Functions

- Validation
- Processing business rules

Diagram



Working

- Accepts request
- Validates data
- Executes business rules
- Stores data
- Returns result

Advantages

- Centralized logic

- Reusable code
- Easy testing
- Better security

Conclusion

Business Logic Layer ensures correct processing before accessing data.

Q8. Cohesion and Coupling

Cohesion

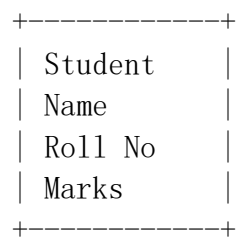
Degree to which related functions belong together.

Coupling

Degree of dependency between modules.

Diagram

High Cohesion



Low Coupling

Student Module

|

Course Module

|

Library Module

(Independent communication)

Benefits

High Cohesion

- Easy maintenance
- Better readability
- Reusability

Low Coupling

- Independent modules
- Easy modification
- Flexible design

Conclusion

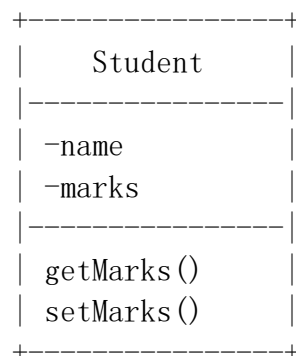
High cohesion and low coupling produce maintainable and reusable software.

Q9. Encapsulation

Definition

Encapsulation binds data and methods into one unit while protecting object data.

Diagram



Encapsulation vs Data Hiding

Encapsulation

Combines data and methods

Achieved using classes

Data Hiding

Restricts direct access

Achieved using private variables

Example

```
private int balance;
```

```
public void deposit()
```

```
public int getBalance()
```

Advantages

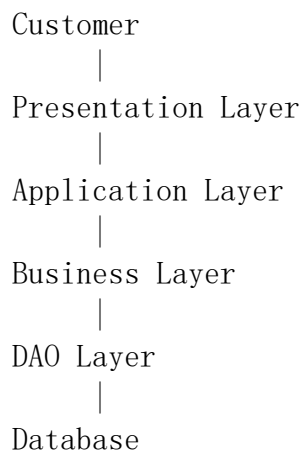
- Security
- Data protection
- Better maintenance
- Controlled access

Conclusion

Encapsulation protects object state while data hiding prevents unauthorized access.

Q10. Online Banking Layered Architecture

Diagram



Design Patterns

- MVC
- DAO
- Singleton
- Factory
- Strategy

Security

- Authentication
- Authorization
- Encryption

Scalability

- Independent layers
- Easy deployment

Maintainability

- Loose coupling
- High cohesion
- Reusable components

Conclusion

Layered architecture improves banking system security, scalability, and maintainability.

Q11. Infrastructure Layer

Definition

Infrastructure Layer provides technical services required by applications.

Responsibilities

- Database connectivity
- External API communication

Diagram

Business Layer

|

Infrastructure Layer

|

| Database | APIs |

Services

- Logging
- Email
- File handling
- Authentication

- Cloud services

Advantages

- Centralized technical services
- Reusable infrastructure
- Easy maintenance

Conclusion

Infrastructure Layer supports all technical operations while keeping business logic clean.

Q12. Singleton Design Pattern

Definition

Singleton ensures only one object of a class exists.

Diagram

Application

|

Singleton Object

|

Shared Resource

Characteristics

- Single instance
- Global access point
- Private constructor

Uses

- Logger
- Configuration Manager
- Database Connection

Limitation

- Difficult unit testing
- Can become global variable

Conclusion

Singleton efficiently manages shared resources while controlling object creation.

Q13. Dependency Injection

Definition

Dependency Injection provides required objects from outside instead of creating them internally.

Diagram

Without DI

Service

|

Repository

With DI

Injector

|

Service

|

Repository

Benefits

- Loose coupling
- Easy testing
- Reusable code
- Better flexibility

Conclusion

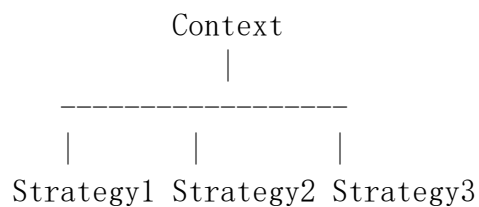
Dependency Injection improves maintainability by reducing direct dependencies.

Q14. Strategy Design Pattern

Definition

Strategy Pattern allows selecting algorithms at runtime.

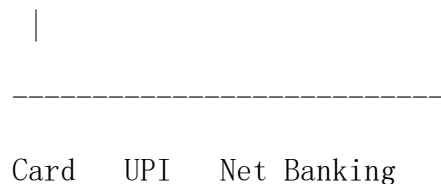
Diagram



Example

Payment System

Payment



Roles

Context

- Uses strategy

Strategy

- Implements algorithm

Advantages

- Flexible

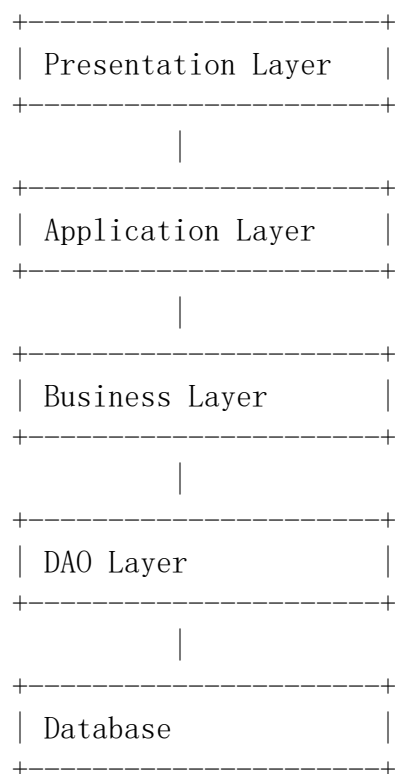
- Open for extension
- Easy maintenance

Conclusion

Strategy Pattern enables runtime selection of different algorithms without changing client code.

Q15. Library Management System

Layered Architecture



Recommended Design Patterns

- MVC
- DAO
- Singleton
- Factory
- Strategy

Features

Presentation Layer

- Login
- Search Books

Business Layer

- Issue Book
- Return Book
- Fine Calculation

DAO Layer

- Book Database
- Member Database

Advantages

- Flexible architecture
- Easy maintenance
- High security
- Code reusability
- Scalability

Conclusion

A layered architecture with MVC, DAO, Singleton, Factory, and Strategy patterns provides a secure, scalable, and maintainable Library Management System. Drawing the layered diagram and briefly explaining each layer in the exam will help secure full marks.